

1. Binary, Octal and Hexadecimal Numbers system & their Conversion

Number System:-

A number system is a set of symbols used to ~~represent~~ represent values derived from a common base or radix(r).

Different number systems:

1. Binary number system (base or radix 2)

It has only 2 digits that are 0 and 1. Every numbers (value) represents with 0 and 1 in this number system.

The base of binary number system is 2, because it has only two digits.

eg. $(110010010)_2$

2. Octal Number System (base 8)

It has only 8 digits from 0 to 7. Every number (value) represents with 0, 1, 2, 3, 4, 5, 6 and 7 in this number system. The base of octal number system is 8, because it has only 8 digits.

For example $(12567)_8$

3. Decimal Number System (base 10)

It has only 10 digits from 0 to 9. Every value (number) represents with 0, 1, 2, 3, 4, 5, 6, 7, 8 & 9 in this number system. The base of decimal number system is 10, because it has only 10 digits.

eg. $(127839)_{10}$

4. Hexadecimal Number System (base 16)

It has 16 alphanumeric value from 0 to 9 and A to F.

Every number (value) represent with 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F in this number system.

The base of hexadecimal number system is 16, because it has 16 alphanumeric values.

Here $A=10$, $B=11$, $C=12$, $D=13$, $E=14$ & $F=15$

eg.

$(178AFD)_{16}$

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12

4. Hexa decimal Number System (base 16)

It has 16 alphanumeric value from 0 to 9 and A to F.

Every number (value) represent with 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F in this number system.

The base of hexadecimal number system is 16, because it has 16 alphanumeric values.

Here $A=10$, $B=11$, $C=12$, $D=13$, $E=14$ & $F=15$

eg.

$(178AFD)_{16}$

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12

19	10011	23	13
20	10100	24	14
21	10101	25	15
22	10110	26	16
23	10111	27	17
24	11000	30	18
25	11001	31	19
26	11010	32	1A
27	11011	33	1B
28	11100	34	1C
29	11101	35	1D
30	11110	36	1E
31	11111	37	1F
32	100000	40	20
33	100001	41	21

19	10011	23	13
20	10100	24	14
21	10101	25	15
22	10110	26	16
23	10111	27	17
24	11000	30	18
25	11001	31	19
26	11010	32	1A
27	11011	33	1B
28	11100	34	1C
29	11101	35	1D
30	11110	36	1E
31	11111	37	1F
32	100000	40	20
33	100001	41	21

Base Conversion Method / Number System Conversion

1. Decimal Number System to other Base

- Decimal to binary Conversion
- Decimal to octal
- Decimal to hexadecimal

2. Other Base to Decimal Number System

- Binary to Decimal
- Octal to Decimal
- Hexadecimal to Decimal

3. Other Base to other Base

- Binary to octal and Vice-versa
- Binary to Hexadecimal Vice-versa
- Octal to Hexadecimal Vice-versa

Decimal Number system to other Base

Step 1:- Divide the given number (Decimal number) by the base of target base system (in which you want to convert the number : Binary (2), Octal (8) and Hexadecimal (16)).

Step 2:- Write the remainder from most significant Bit (MSB) to least significant Bit (LSB) step as MSB or Bottom to up.

(or)

The 1's and 0's written as remainder in reverse order (i.e. bottom to top) is the required number.

Decimal to Binary :-

Divide given decimal number by 2 and write remainder from MSB to LSB.

Ex.1. $(47)_{10}$ to $()_2$

$$\begin{array}{r}
 2 \overline{) 47} \text{ --- } 1 \\
 2 \overline{) 23} \text{ --- } 1 \quad \uparrow \\
 2 \overline{) 11} \text{ --- } 1 \\
 2 \overline{) 5} \text{ --- } 1 \\
 2 \overline{) 2} \text{ --- } 0 \\
 2 \overline{) 1} \text{ --- } 1 \\
 0
 \end{array}$$

$$\therefore (47)_{10} = (101111)_2$$

Ex.2. $(78)_{10}$ to $()_2$

$$\begin{array}{r}
 2 \overline{) 78} \text{ --- } 0 \\
 2 \overline{) 39} \text{ --- } 1 \\
 2 \overline{) 19} \text{ --- } 1 \\
 2 \overline{) 9} \text{ --- } 1 \\
 2 \overline{) 4} \text{ --- } 0 \\
 2 \overline{) 2} \text{ --- } 0 \\
 2 \overline{) 1} \text{ --- } 1 \\
 0
 \end{array}$$

$$\therefore (78)_{10} = (1001110)_2$$

Ex.3. $(13)_{10} = ()_2$

Quotient	Remainder
$13 \div 2 = 6$	1 $\uparrow$
$6 \div 2 = 3$	0
$3 \div 2 = 1$	1
$1 \div 2 = 0$	1

$$\therefore (13)_{10} = (1101)_2$$

Ex.4. $(25.625)_{10}$

For integer part,

$$\begin{array}{r}
 2 \overline{) 25} \text{ --- } 1 \\
 2 \overline{) 12} \text{ --- } 0 \quad \uparrow \\
 2 \overline{) 6} \text{ --- } 0 \\
 2 \overline{) 3} \text{ --- } 1 \\
 2 \overline{) 1} \text{ --- } 1 \\
 0
 \end{array}
 \quad \therefore (25)_{10} = (11001)_2$$

For Fractional Part

$$\begin{array}{r}
 0.625 \times 2 = 1.25 \text{ --- } 1 \\
 0.25 \times 2 = 0.5 \text{ --- } 0 \quad \downarrow \\
 0.5 \times 2 = 1.0 \text{ --- } 1 \\
 \therefore (0.625)_{10} = (0.101)_2
 \end{array}$$

$$\therefore (25.625)_{10} = (11001.101)_2$$

Ex.5. $(3.703125)_{10}$ to $()_2$

For integer part,

$$\begin{array}{r} 2 \overline{) 3} \text{ --- } 1 \uparrow \\ 2 \overline{) 1} \text{ --- } 1 \uparrow \\ \hline 0 \end{array}$$

$$\therefore (3)_{10} = (11)_2$$

For Fractional part,

$$0.703125 \times 2 = 1.40625 \text{ --- } 1$$

$$0.40625 \times 2 = 0.8125 \text{ --- } 0$$

$$0.8125 \times 2 = 1.625 \text{ --- } 1$$

$$0.625 \times 2 = 1.25 \text{ --- } 1$$

$$0.25 \times 2 = 0.5 \text{ --- } 0$$

$$0.5 \times 2 = 1.0 \text{ --- } 1$$

$$\therefore (3.703125)_{10} = (11.101101)_2$$

Extra Question

a. $(37.432)_{10} = ()_2$

b. $(24.625)_{10} = ()_2$

c. $(64)_{10} = ()_2$

d. $(0.625)_{10} = ()_2$

e. $(23.6)_{10} = ()_2$

Decimal to octal Conversion :-

Divide given decimal number by 8 and write remainder from MSB to LSB (Bottom to Top)

Ex. 1. $(100)_{10}$ to $()_8$

$$\begin{array}{r} 8 \overline{) 100} \text{ --- } 4 \\ 8 \overline{) 12} \text{ --- } 4 \\ 8 \overline{) 1} \text{ --- } 1 \\ 0 \end{array} \quad \uparrow$$

$$\therefore (100)_{10} = (144)_8$$

Ex. 2. $(426)_{10}$ to $()_8$

$$\begin{array}{r} 8 \overline{) 426} \text{ --- } 2 \\ 8 \overline{) 53} \text{ --- } 5 \\ 8 \overline{) 6} \text{ --- } 6 \\ 0 \end{array} \quad \uparrow$$

$$\therefore (426)_{10} = (652)_8$$

Ex. 3.

$(100.342)_{10}$ to $()_8$

For integer value

$$\therefore (100)_{10} = (144)_8$$

For fraction value

$$(0.342)_{10} = ()_8$$

$$0.342 \times 8 = 2.736 \text{ --- } 2$$

$$0.736 \times 8 = 5.888 \text{ --- } 5$$

$$0.888 \times 8 = 7.104 \text{ --- } 7$$

$$0.104 \times 8 = 0.832 \text{ --- } 0$$

$$\therefore (0.342)_{10} = (0.2570)_8$$

$$\therefore (100.342)_{10} = (144.2570)_8$$

Ex. 4. $(0.96)_{10}$ to $()_8$

$$0.96 \times 8 = 7.68 \text{ --- } 7$$

$$0.68 \times 8 = 5.44 \text{ --- } 5$$

$$0.44 \times 8 = 3.52 \text{ --- } 3$$

$$0.55 \times 8 = 4.16 \text{ --- } 4$$

$$\therefore (0.96)_{10} = (0.7534)_8$$

Other Extra Question

a. $(405)_{10} = ()_8$

b. $(0.0625)_{10} = ()_8$

c. $(175)_{10} = ()_8$

d. $(0.23)_{10} = ()_8$

e. $(13.41)_{10} = ()_8$

f. $(1341)_{10} = ()_8$

g. $(126)_{10} = ()_8$

h. $(1024)_{10} = ()_8$

Decimal to Hexadecimal :-

Decimal given decimal number by 16 and write remainder from MSB to LSB (Bottom to top).

Ex.1, $(3479)_{10}$ to $()_{16}$

$$\begin{array}{r} 16 \overline{) 3479} - 7 = 7 \\ 16 \overline{) 217} - 9 = 9 \\ 16 \overline{) 13} - 13 = D \\ 0 \end{array} \quad \uparrow$$

$$\therefore (3479)_{10} = (D9F)_{16}$$

Ex.2 $(3479.565)_{10} = ()_{16}$

For integer number

$$(3479)_{10} = (D9F)_{16}$$

For fraction number

$$(0.565)_{10} \text{ to } ()_{16}$$

$$0.565 \times 16 = 9.04 \text{ — } 9$$

$$0.04 \times 16 = 0.64 \text{ — } 0$$

$$0.64 \times 16 = 10.24 \text{ — } 10 = A$$

$$0.64 \times 16 = 3.84 \text{ — } 3$$

$$0.84 \times 16 = 13.44 \text{ — } 13 = D$$

$$0.44 \times 16 = 7.04 \text{ — } 7$$

$$0.04 \times 16 = 0.64 \text{ — } 0$$

$$\therefore (0.565)_{10} = (0.90A3D70)_{16}$$

$$\therefore (3479.565)_{10} = (D9F.90A3D70)_{16}$$

Ex.3 $(495.62)_{10}$ to $()_{16}$

Part.1st

$$\begin{array}{r} 16 \overline{) 495} - 15 = F \\ 16 \overline{) 30} - 14 = E \\ 16 \overline{) 1} - 1 = 1 \\ 0 \end{array} \quad \uparrow$$

$$\therefore (495)_{10} = (1EF)_{16}$$

Part.2nd

$$0.62 \times 16 = 9.92 \text{ — } 9$$

$$0.92 \times 16 = 14.72 \text{ — } 14 = E$$

$$0.72 \times 16 = 11.52 \text{ — } 11 = B$$

$$0.52 \times 16 = 8.32 \text{ — } 8$$

$$0.32 \times 16 = 5.12 \text{ — } 5$$

$$0.12 \times 16 = 1.92 \text{ — } 1$$

$$0.92 \times 16 = 14.72 \text{ — } 14 = E$$

$$\therefore (0.62)_{10} = (0.9EB851E)_{16}$$

$$\therefore (495.62)_{10} = (1EF.9EB851E)_{16}$$

* Convert $()_{10}$ to $()_{16}$

a. $(1341)_{10}$

b. $(3215.3)_{10}$

c. $(65536)_{10}$

d. $(2479)_{10}$

e. $(348.37)_{10}$

To convert Number system

Step A. Determine the base value of source number system (that you want to convert) and also determine the position of digits from LSB (First digit's position - '0', second digit's position - '1' and so on).

Step B:- Multiply each digit with its corresponding multiplication of position value and Base of source number system's Base.

Step C:- Add the resulted value of step B.

a. Binary to Decimal

Step 1: Write binary digits as power of 2 increasing from right to left starting from 0.

Step 2: Convert each power of two into its decimal equivalent term.

Step 3: Add these terms to give the decimal number.

Ex. 1. $(101)_2$ to $()_{10}$

Solⁿ

$$\begin{aligned} 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 &= 1 \times 4 + 0 \times 2 + 1 \times 1 \\ &= 4 + 0 + 1 \\ &= 5 \end{aligned}$$

$$\therefore (101)_2 = (5)_{10}$$

For Fractional

→ Write binary numbers as the negative power of 2 from left to right starting from point -1, -2 & so on.
→ then same as integer step 2 & step 3.

Ex. 2.

$$(101.110)_2 \text{ to } ()_{10}$$

$$= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}$$

$$= 4 + 0 + 1 + 0.5 + 0.25 + 0$$

$$= (5.875)_{10}$$

Ex. 3. $(1110.001)_2$

$$= 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$= 8 + 4 + 2 + 0 + 0 + 0 + 0.125$$

$$= (14.125)_{10}$$

Questions:-

a. $(11001)_2 = ()_{10}$

b. $(1010101010)_2 = ()_{10}$

c. $(11110)_2 = ()_{10}$

d. $(100001)_2 = ()_{10}$

e. $(110.101)_2 = ()_{10}$

Octal to decimalEx. 1. $(274)_8$ to $()_{10}$

$$\begin{aligned}
 & 2 \times 8^2 + 7 \times 8^1 + 4 \times 8^0 \\
 & = 128 + 56 + 4 \\
 & = (188)_{10}
 \end{aligned}$$

Ex. 2. $(274.267)_8$ to $()_{10}$

$$\begin{aligned}
 & 2 \times 8^2 + 7 \times 8^1 + 4 \times 8^0 + 2 \times 8^{-1} + 6 \times 8^{-2} + 7 \times 8^{-3} \\
 & = 128 + 56 + 4 + 0.25 + 0.094 + 0.014 \\
 & = (188.358)_{10}
 \end{aligned}$$

Ex. 3 $(347)_8$ to $()_{10}$

$$\begin{aligned}
 & 3 \times 8^2 + 4 \times 8^1 + 7 \times 8^0 \\
 & = 3 \times 64 + 4 \times 8 + 7 \\
 & = (231)_{10}
 \end{aligned}$$

Questiona. $(304)_8$ to $()_{10}$ b. $(1534)_8$ c. $(425.24)_8$ d. $(75.56)_8$ e. $(2176.45)_8$ Ex. 4. $(521)_8$ to $()_{10}$

$$\begin{aligned}
 & 5 \times 8^2 + 2 \times 8^1 + 1 \times 8^0 \\
 & = 320 + 16 + 1 \\
 & = (337)_{10}
 \end{aligned}$$

Ex. 5 $(127.54)_8$ to $()_{10}$

$$\begin{aligned}
 & = 1 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 + 5 \times 8^{-1} + 4 \times 8^{-2} \\
 & = 64 + 16 + 7 + 0.625 + 0.0625 \\
 & = (87.6875)_{10}
 \end{aligned}$$

Hexadecimal to DecimalEx. 1.

$$(345)_{16} \text{ to } ()_{10}$$

$$= 3 \times 16^2 + 4 \times 16^1 + 5 \times 16^0$$

$$= 768 + 64 + 5$$

$$= (837)_{10}$$

Ex. 2 $(2B.C4)_{16} \text{ to } ()_{10}$

$$= 2B.C4$$

$$= 2 \times 16^1 + 11 \times 16^0 + 12 \times 16^{-1} + 4 \times 16^{-2}$$

$$= 32 + 11 + 0.75 + 0.015625$$

$$= (43.765625)_{10}$$

Question:-

a) $(865F)_{16} \text{ to } (?)_{10}$

b) $(B8C)_{16}$

c) $(AA)_{16}$

d) $(BABE.C8)_{16}$

e) $(CD.FA)_{16}$

Binary to Octal:-Method 1

Step 1:- Separate the given binary in the group of 3 digits from LSB.

Step 2:- Find the corresponding value of binary digits.

Step 3:- Write the value from MSB to LSB.

Ex. 1. $(1011010)_2$ to $()_8$

$$\begin{array}{ccc} 1 & 011 & 010 \\ \hline 1 & 3 & 2 \end{array}$$

$$(1011010)_2 = (132)_8$$

Que:

Ⓐ $(10111)_2$ to $()_8$

Ⓑ $(01101)_2$

Ⓒ $(1001.101)_2$

Ⓓ $(1110.011)_2$

Ex. 2 $(11001.11)_2$ to $()_8$

$$\begin{array}{ccc} 0 & 1100 & 1.110 \\ \hline \downarrow & \downarrow & \downarrow \\ 3 & 1 & 6 \end{array}$$

$$(11001.11)_2 = (31.6)_8$$

Method 2

Step 1. Convert given binary number into decimal.

Step 2. Again Convert this decimal number to octal number.

Ex. 1.

$$(101)_2$$

$$= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 5$$

$$\therefore (101)_2 = (5)_{10}$$

Again

$$\begin{array}{r} 8 \overline{) 5} - 5 \\ \hline 0 \end{array}$$

$$\therefore (5)_{10} = (5)_8$$

Octal to BinaryMethod 1

- ① Take each digit individual, assuming it to be decimal digit.
- ② Convert it into binary and place it in the combination of 3 bit each.

eg. ① $(27)_8 = ()_2$

$$2 = 010$$

$$7 = 111$$

$$(27)_8 = (010111)_2$$

Ex. 2 $(1745.246)_8 \rightarrow ()_2$

1	7	4	5	2	4	6
001	111	100	101	010	100	110

$$= (001111100101010100110)_2$$

$$= (1111100101.010100110)_2$$

Method 2

- ① Convert octal to decimal
- ② Again decimal to binary

Questions :-

① $(21)_8 \rightarrow ()_2$

② $(2746)_8 \rightarrow ()_2$

③ $(35.346)_8$

④ $(257.43)_8$

⑤ $(75.3)_8$

Binary to Hexadecimal

Method 1:-

Step 1:- Separate the given binary in the group of 4 digits from LSB.

Step 2:- Find the corresponding value of Binary digits.

Step 3:- Write value from MSB to LSB.

Ex.1 $(11111011101110010)_2$ to $()_{16}$

0001 1111 0111 0111 0010

1 15 7 7 2

F

$(1F772)_{16}$

Method 2:-

Step 1. Binary to decimal

Step 2. decimal to Hexadecimal

Question

Q) $(11011110111)_2$ to $()_{16}$

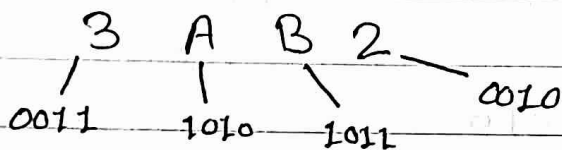
Hexadecimal to Binary:-

step 1: separate each digit of given number.

step 2: find binary value of given number.

step 3: write value from MSB to LSB.

Ex.1 $(BAB2)_{16}$ to $()_2$



$$(3AB2)_{16} = (0011\ 1010\ 1011\ 0010)_2$$

$$= (11101010110010)_2$$

Octal to Hexadecimal:-

Method 1:-

Convert Octal to Decimal & Decimal to Hexadecimal.

Method 2:-

Convert octal to Binary & Binary to Hexadecimal.

Hexadecimal to octal:-

Method 1:-

Convert Hexadecimal to Decimal & Decimal to Octal.

Method 2:-

Convert Hexadecimal to Binary & Binary to octal.

Arithmetic operation with binary number or any number with base 'r' follow the same rules as for decimal number. Because binary number system is also the positional number system, the rules for the positional number apply to the binary numbers also.

1. Binary addition:-

As with decimal addition, to add two binary numbers we need to add two corresponding bits from the two numbers at a time. Following are the rules for binary addition.

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10 \text{ (with sum 0 \& carry 1)}$$

For Example:-

$$\begin{array}{r} \text{Carry} \rightarrow \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \\ \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \\ + 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \\ \hline \text{Sum} \rightarrow \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \end{array}$$

2. Binary Subtraction:-

Similar to decimal subtraction, to subtract a binary numbers from other we need to subtract a corresponding bit from other. Following are the rules for binary subtraction.

$$0 - 0 = 0$$

$$0 - 1 = 1 \text{ (with borrowing 1)}$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

Example 1:- borrow $\rightarrow$ 0 1 1 0

1 1 1 $\xrightarrow{10}$ 0 $\xrightarrow{10}$ 1 $\leftarrow$ minuend

- 0 1 0 0 1 0 $\leftarrow$ Subtrahend

difference $\rightarrow$ 1 0 0 1 1 1

Ex 2.

1 0 1 1 0 1 1

- 1 0 0 1 0

1 0 0 1 0 0 1

Ex 3

$\xrightarrow{10}$ 1 $\xrightarrow{10}$ 1 $\xrightarrow{10}$ 0 $\xrightarrow{10}$ 1 1 1

- 0 1 1 0 1 1 1 0

0 1 1 0 0 1 0 1

3. Binary Multiplication:-

Following are the rules for binary multiplication.

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

eg. 1. 1011×1011

1011 $\rightarrow$ multiplicand

$\times$ 1011 $\rightarrow$ Multiplier.

1011

1011 $\times$

0000 $\times \times$

$\times$ 1011 $\times \times \times$

1011001 $\rightarrow$ product

4. Binary division :-

Rules:- $0 \div 0 = \text{not defined}$

$$0 \div 1 = 0$$

 $1 \div 0 = \text{not defined}$

$$1 \div 1 = 1$$

eg. Divide 101011 by 110

$$\begin{array}{r}
 110 \overline{) 101011} \quad 111 \leftarrow \text{Quotient} \\
 \underline{- 110} \\
 1001 \\
 \underline{- 110} \\
 111 \\
 \underline{- 110} \\
 1 \leftarrow \text{Remainder}
 \end{array}$$

eg.2.

$$\begin{array}{r}
 (5) \quad (26) \quad (5) \\
 101 \overline{) 11010} \quad 101 \\
 \underline{- 101} \\
 11 \\
 \underline{- 00} \\
 110 \\
 \underline{- 101} \\
 1 \\
 (1)
 \end{array}$$

Q. What are the types of Complement in Subtraction

20

Subtracting using Complement method (9's, 10's, 1's, 2's)

Computer's Complement short Note.

Subtraction using Complement method

Complement :-

In computer system, Subtraction is not performed directly as arithmetic subtraction. It is performed by the technique called Complement. It is the process of repeated addition.

There are two types of Complements :-

i) r 's Complement

ii) $(r-1)$'s Complement

Where r is the base of a number system. In binary number system, there are two types of Complements

2's Complement and 1's Complement. Similarly, decimal

number system has 10's and 9's Complement.

Binary no. system

decimal No. system

1. 9's Complement :-

The 9's Complement of decimal number can be obtained by subtracting each digit of the number from 9.

For example :- 9's complement of 3 is $(9-3) = 6$

9's Complement of 36 is $(99-36) = 63$

9's complement of 234 is $(999-234) = 765$

2. 10's Complement :-

The 10's Complement of decimal number can be obtained by adding 1 to the least significant digit of 9's Complement of the number.

For example: 10's complement of 3 is 7 ($9-3 = 6+1 = 7$)

& 10's complement of 123 is 877 ($999-123 = 876+1 = 877$)

3. 1's Complement :-

1's Complement of binary number is obtained by subtracting each bit by 1. We can get complement by simply replacing 1 by 0 and 0 by 1.

For example :-

$$1's \text{ Complement of } 1011 = 0100$$

(or)

$$\begin{array}{r} 1011 \\ - 1111 \\ \hline 0100 \end{array}$$

4. 2's Complement :-

2's Complement of a binary number is obtained by adding binary 1 to the 1's Complement of the number.

For example.

2's Complement of 1101101 is :-

$$1's \text{ Complement of } 1101101 = 0010010$$

$$2's \text{ complement of } 1101101 = 0010010 + 1$$

$$= 0010011$$

Decimal	Binary
9's $\rightarrow$ 999	1's $\rightarrow$ replace 0 by 1 & 1 by 0
पहिले 9s बनाउने then 10's $\rightarrow$ add 1.	पहिले 1's बनाउने then add 1 2's $\rightarrow$

Subtraction of decimal numbers by using 9's Complement :-

Step:-

- Make the both number having same number of digits.
- Determine the 9's complement of the number to be subtracted (Subtrahend).
- Add the 9's complement to the given number from which we subtract (minuend).
- If there exist's any additional digit (carry) in the result after addition, remove it and add it to the remaining digits to find the final result. If there exist's no any carry then determine the 9's complement of the result and prefix by negative sign to get the final result.

i) Subtracting small from greater number.

For example:- Subtract $(123)_{10}$ from $(345)_{10}$

Solⁿ

9's Complement of 123 = $(999 - 123) = 876$

Adding it with minuend $(345) =$

$+ 345$

1|221

In the result, there exist one carry i.e. 1 so, adding this carry to the remaining digit.

221

+ 1

222

Hence, $(222)_{10}$ is the required result after subtracting $(123)_{10}$ from $(345)_{10}$.

ii) Subtracting greater number from smaller number.
example:-

Subtracting $(345)_{10}$ from $(123)_{10}$

Solⁿ:

$$9's \text{ Complement of } 345 = (999 - 345) = 654$$

Adding it with minuend $(123) \Rightarrow 654$

$$\begin{array}{r} + 123 \\ \hline 777 \end{array}$$

In the result, there doesn't exist any carry. So calculating 9's Complement of the result & prefix by (-ve) sign.

$$9's \text{ Complement of } 777 = (999 - 777) = 222$$

Hence, $(-222)_{10}$ is the required result after subtracting $(345)_{10}$ by $(123)_{10}$.

Subtraction of decimal numbers by using 10's Complement:-

Steps:

- Make the same numbers having same number of digits.
- Determine the 10's Complement of the number to be subtracted (subtrahend).
- Add the 10's Complement to the given number from which we subtract (minuend).
- If there exists any additional digit (carry) in the result after addition, remove it from the result and the remaining digits form is the final result. If there exists no any carry then determine the 10's complement of the result and prefix by negative sign to get the final result.

i) Subtracting Smaller number from greater number.

Example:-

Subtract $(123)_{10}$ from $(345)_{10}$

Soln:

$$\begin{aligned} \text{10's Complement of } 123 &= (999 - 123) = 876 + 1 \\ &= 877 \end{aligned}$$

$$\begin{array}{r} \text{Adding it with minuend } (345) \quad 877 \\ + 345 \\ \hline 1222 \end{array}$$

In this result, there exist one carry i.e., 1 so, removing it to find the result.

Hence, $(222)_{10}$ is the required result after subtracting $(123)_{10}$ from $(345)_{10}$.

ii) Subtracting greater number from smaller number.

For example:-

$(345)_{10}$ from $(123)_{10}$

Solⁿ.

$$10's \text{ Complement of } 345 = 999 - 345$$

$$= 654 + 1$$

$$= 655$$

$$\begin{array}{r} \text{Adding it with minuend } (123) \quad 655 \\ + 123 \\ \hline 778 \end{array}$$

In the result there doesn't exist any Carry, So calculating 10's Complement of the result and prefix by negative sign.

$$10's \text{ Complement of } 778 = 999 - 778$$

$$= 221 + 1$$

$$= 222$$

Hence, $(-222)_{10}$ is the required result after subtracting $(345)_{10}$ by $(123)_{10}$.

Subtracting of binary number using 1's Complement

Step:-

- ① Make the both numbers having same number of bits.
- ② Determine the 1's complement of the number to be subtracted.
- ③ Add the 1's complement to the given number from which we subtract (minuend).
- ④ If there exists any additional bit (carry) in the result after addition, remove and add it to the result. If there exists no carry determine the 1's complement of the result and prefix by negative sign to get the final result.

i) Subtracting Small number from Larger number.

Example:-

Subtract $(110000)_2$ from $(111000)_2$

sol:-

1's Complement of $110000 = 001111$

Adding it with minuend $(111000) + 111000$

1 000111

In the result, there exist one carry i.e. 1 so adding this carry to the remaining digit.

$$\begin{array}{r} 000111 \\ + 1 \\ \hline 001000 \end{array}$$

Hence, $(001000)_2$ is the required result after Subtracting $(110000)_2$ from $(111000)_2$.

ii) Subtracting Greater number from smaller number.

Example:-

Subtract $(111000)_2$ from $(110000)_2$

1's Complement of $111000 \rightarrow 000111$

Adding it with minuend (110000)

$+ 110000$

110111

In this result, there doesn't exist any carry, so, calculating 1's complement of the result and prefix by negative sign.

1's Complement of $110111 = 001000$

Hence $(-001000)_2$ is the required result after subtracting $(111000)_2$ from $(110000)_2$

Subtracting of binary number using 2's Complement.

Steps:-

- ① Make the both numbers having same number of bits.
- ② Determine the 2's Complement of the number to be subtracted.
- ③ Add the 2's Complement to the given number from which we subtracted (minuend).
- ④ If there exists any additional bit (carry) in the result after addition, remove it from the result and the remaining digits form is the final result.

If there exist's no carry determine the 2's complement of the result and prefix by negative sign to get the final result.

i> Subtracting Smaller number from greater number, For example:-

Subtract $(110000)_2$ from $(111000)_2$

Solⁿ:

1's Complement of $110000 = 001111$

2's Complement of $110000 = 001111$

+ 1

010000

Adding it with minuend

+ 111000

1001000

In this result, there exist one carry i.e. 1. So, removing the carry.

Hence, $(001000)_2$ is the required result after subtracting $(110000)_2$ from $(111000)_2$.

Subtracting of binary number using 2's Complement, Steps:-

- ① Make the both numbers having same number of bits.
- ② Determine the 2's Complement of the number to be Subtracted.
- ③ Add the 2's Complement to the given number from which we subtracted (minuend).
- ④ If there exists any additional bit (Carry) in the result after addition, remove it from the result and the remaining digits form is the final result.

If there exist's no carry determine the 2's Complement of the result and prefix by negative sign to get the final result.

i> Subtracting Smaller number from greater number, for example:

Subtract $(110000)_2$ from $(111000)_2$

Sol:

1's Complement of $110000 = 001111$

2's Complement of $110000 = 001111$

+ 1

0 10000

Adding it with minuend

+ 111000

1 001000

In this result, there exist one carry i.e. 1. So, removing the carry.

Hence, $(001000)_2$ is the required result after subtracting $(110000)_2$ from $(111000)_2$.

ii) Subtracting Greater number from smaller number.

Example:-

Subtract $(111000)_2$ from $(110000)_2$

Solⁿ:

1's Complement of 111000 000111

2's Complement of 111000 000111

+ 1

001000

Adding it with minuend (110000)

+ 110000

111000

In the result, there doesn't exist carry, so, Calculating 2's complement of the result and prefix by negative (-ve) sign.

1's Complement of 111000 000111

2's Complement of 111000 000111

+ 1

001000

Hence, (-001000) is the required result after subtracting $(111000)_2$ from $(110000)_2$

Imp. Questions :-

9's and 10's Complement.

① $571.52 - 837.81$

9's Complements of Subtraction

Step 1.

$$999.99 - 837.81 = 162.18$$

Step 2.

$$571.52 + 162.18 = 733.70$$

Since, there is no carry so the final answer is negative. To get the final answer we take 9's complements of 733.70

Step 3.

$$999.99 - 733.70 = 266.29$$

 -266.29 is answer.

10's Complements of Subtraction.

Step 1.

$$999.99 - 837.81 = 162.18$$

$$162.18 + 1 = 162.19$$

Step 2. $571.52 + 162.19$
 $= 733.71$

Since, there is no carry final answer is negative

Step 3.

10's is

$$999.99 - 733.71 = 266.28$$

$$266.28 + 1 = 266.29$$

 -266.29 is answer.

⑥ $453.35 - 321.17$

Date

Page 31

9's Complements

10's Complement

$$999.99 - 321.17 = 678.82$$

$$999.99 - 321.17 = 678.82$$

$$453.35$$

$$678.82$$

$$+ 678.82$$

$$+ 1$$

$$1132.17$$

$$678.83$$

Since there exist Carry

Adding carry 132.17

$$+ 1$$

$$(132.18)_{10}$$

Now,

$$453.35$$

$$+ 678.83$$

$$1132.18$$

Since there exist Carry

Remove Carry $(132.18)_{10}$

Answer

1's & 2's Complement

Q. $1010.110 - 101.101$

$$1010.110 - 0101.101$$

Solⁿ

1's Complement 1010.010

of 0101.101 ~~1010~~

Adding it with minuend (1010.110)

$$\begin{array}{r} 1010.010 \\ + 1010.110 \\ \hline 10101.000 \end{array}$$

there exist one carry i.e. 1 So adding this carry to the remaining digit.

$$0101.000 \leftarrow$$

$$\begin{array}{r} \rightarrow + 1 \\ \hline 0101.001 \end{array}$$

Hence, $(101.001)_2$ is required result.

2's Complement 1's Complement + 1

$$1010.010$$

$$+ 1$$

$$\hline 1010.011$$

Adding it with minuend (1010.110) $+ 1010.110$

$$\begin{array}{r} 1010.011 \\ + 1010.110 \\ \hline 10101.001 \end{array}$$

there exist one carry i.e. 1. So, removing the carry.

Hence, $(101.001)_2$ is required result.

Binary Code: (Coding)

→ It represents text, computer process instructions or any other data using a two-symbol system (two stable states i.e. 0 or 1).

→ It is the symbolic representation of discrete information which may be present in the form of numbers, letters or physical quantities.

→ Codes are classified as

i) Weighted Codes (BCD)

ii) Non-weighted Codes (Gray, Excess-3)

iii) Alphanumeric Codes (Alphanumeric Codes)

ASCII - American Standard Code for Information Interchange.

BCD (Binary-Coded Decimal) Codes:-

→ In this decimal digit 0 to 9 are represented by their natural binary equivalent using 4 bits.

→ It is a straight assignment of binary equivalent to each digit.

→ The BCD code is the 8, 4, 2, 1 code.

→ This code only encodes the first ten values from 0 to 9 (i.e. 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000 & 1001)

→ Each decimal digit is coded separately by 4 bits.

$$\text{eg } (325)_{10} = (\underbrace{0011}_3 \underbrace{0010}_2 \underbrace{0101}_5)_{\text{BCD}}$$

$$\text{⑥ } (856)_{10} = (\quad)_{\text{BCD}}$$

$$\text{⑦ } (95)_{10} = (10111111)_2 = (1001 \ 0101)_{\text{BCD}}$$

Binary Code: (Coding)

→ It represent text Computer process instructions or any other data using a two-symbol system (two stable states i.e. 0 or 1).

→ It is the symbolic representation of discrete information which may be present in the form of numbers, letters or physical quantities.

→ Codes are classified as

i) Weighted Codes (BCD)

ii) Non-weighted Codes (Gray, Excess-3)

iii) Alphanumeric Codes (Alphanumeric Codes)

ASCII - American Standard Code for Information Interchange.

BCD (Binary - Coded Decimal) Codes:-

→ In this decimal digit 0 to 9 are represented by their natural binary Equivalent using 4 bits.

→ It is a straight assignment of binary equivalent to each digit.

→ The BCD Code is the 8, 4, 2, 1 Code.

→ This Code only encodes the first ten value from 0 to 9 (i.e. 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000 & 1001)

→ Each decimal digit is coded separately by 4 bits.

$$\text{eg① } (325)_{10} = (\underbrace{0011}_3 \underbrace{0010}_2 \underbrace{0101}_5)_{BCD}$$

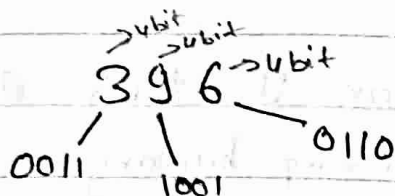
$$\text{② } (856)_{10} = (\quad)_{BCD}$$

$$\text{③ } (95)_{10} = (10111111)_2 \\ = (1001 \ 0101)_{BCD}$$

Decimal	BCD	8	4	2	1	
0	0000	0	0	0	0	= 0
1	0001	0	0	0	1	= 1
2	0010	0	0	1	0	= 2
3	0011	0	0	1	1	= 2+1
4	0100	0	1	0	0	= 4
5	0101	0	1	0	1	= 4+1
6	0110	0	1	1	0	= 4+2
7	0111	0	1	1	1	= 4+2+1
8	1000	1	0	0	0	= 8
9	1001	1	0	0	1	= 8+1
10	0001 0000					
11	0001 0001					
12	0010 0010					

→ A number with n decimal digit will require $4n$ bits in BCD.

eg decimal 396 is represented in BCD with 12 bits as 0011 1001 0110



→ Numbers greater than 9 has a representation different from its equivalent binary number, even though both contain 1's or 0's.

→ Binary combinations 1010 through 1111 are not used and have no meaning in the BCD Code.

$$\text{eg. } (185)_{10} = (0001 \ 1000 \ 0101)_{\text{BCD}}$$

$$= (10111001)_2$$

$$2 \overline{185} - 1$$

$$2 \overline{92} - 0$$

$$2 \overline{46} - 0$$

$$2 \overline{23} - 1$$

$$2 \overline{11} - 1$$

$$2 \overline{5} - 1$$

$$2 \overline{2} - 0$$

$$2 \overline{1} - 1$$

Merits :-

- It is easy to encode and decode decimal into BCD and vice versa.
- It is also simple to implement a hardware algorithm for the BCD Converter.
- It is very useful in digital system wherever decimal information is given either as inputs or displayed as outputs.

Demerits :-

- For integers greater than 9, BCD occupies more space than the corresponding binary.

For eg. Decimal $\rightarrow (13)_{10}$

Binary $\rightarrow (1101)_2$

BCD code $\rightarrow (0001 \ 0011)_{\text{BCD}}$

- The arithmetic operations using BCD code require a complex design of arithmetic and logic unit (ALU) than the straight binary number system.

BCD Addition :-Step 1. Add the BCD (Same as Binary)

$0+0=0$

$0+1=1$

$1+0=1$

$1+1=10 \text{ (with Sum 0 \& Carry 1)}$

For eg. Carry $\rightarrow$ 0 1 1 1 1

$$\begin{array}{r}
 101101 \\
 +001111 \\
 \hline
 111100
 \end{array}$$

Sum $\rightarrow$ 1 1 1 1 0 0

Step 2 :- If a 4-bit sum is equal to or less than 9, it is final answer (valid BCD number)Step 3 :- If a 4 bit sum is greater than 9, then it is an invalid result.

Add 6 (0110) to the 4 bit sum.

Step 4 :- If a Carry result when 6 is added simply, add Carry to the next 4 bit group.Example

(a) $0011 + 0100$

0011

$+ 0100$

0111

$$\begin{array}{r}
 1000 \quad 0110 \quad 86 \\
 0001 \quad 0011 \quad +13 \\
 \hline
 1001 \quad 1001 \quad 99
 \end{array}$$

It is less than 9, Hence it is valid answer.

⑥ 15 + 16 (Add using BCD)

$$\begin{array}{r}
 0001 \quad 0101 \\
 + 0001 \quad 0110 \\
 \hline
 0010 \quad 1011 \\
 (i) \quad (ii)
 \end{array}$$

(i) is valid

(ii) is invalid because $(ii) > 9$

Adding 6 (0110) to valid (ii) part.

$$\begin{array}{r}
 1011 \\
 + 0110 \\
 \hline
 1 \quad 0001
 \end{array}$$

Adding carry 0001 to the next (i) part, invalid is 0001

next 4 bit part (i)

$$\begin{array}{r}
 0010 \\
 + \quad 1 \\
 \hline
 0011 < 9
 \end{array}$$

It is valid

Hence,

$$\begin{array}{cc}
 0011 & 0001 \\
 \downarrow & \downarrow \\
 3 & 1
 \end{array}$$

(31)₁₀

$ \begin{array}{r} (i) \quad 450 \\ 417 \\ \hline 867 \end{array} $	$ \begin{array}{r} (ii) \quad 7 \\ + 5 \\ \hline 12 \end{array} $
$ \begin{array}{r} (iii) \quad 19_{10} \\ + 8_{10} \\ \hline 27_{10} \end{array} $	$ \begin{array}{r} 1001 \\ 1000 \\ \hline 10001 < \text{invalid} \\ + 0110 \\ \hline 1 \quad 0111 \\ 0000 \\ + 1 \\ \hline 0001 \end{array} $

② $24 + 17$

$$\begin{array}{r}
 0010 \quad 0100 \\
 + 0001 \quad 0111 \\
 \hline
 0011 \quad 1011 \\
 \text{(i)} \quad \quad \text{(ii)}
 \end{array}$$

(i) is valid

ii) is invalid because $ii > 9$

Adding 6 (0110) to invalid (ii) part

$$\begin{array}{r}
 1011 \\
 + 0110 \\
 \hline
 10001
 \end{array}$$

Adding carry 1 to the next part (i), invalid is 0001 next 4 bit part (i)

$$\begin{array}{r}
 0011 \\
 + 1 \\
 \hline
 0100 \quad \text{9}
 \end{array}$$

It is valid, Hence, $0100 \quad 0001$
 $4 \quad 1$

$(41)_{10}$

Ans.

$$\textcircled{a} (97)_{10} + (48)_{10}$$

Adding Using BCD

$$\begin{array}{r} 1001 \quad 0111 \\ 0100 \quad 1000 \\ \hline 1101 \quad 1111 \\ \text{(i)} \quad \text{(ii)} \end{array}$$

Hence, Both part are invalid.

Now, adding 6 (0110) to (ii) invalid part.

$$\begin{array}{r} 1111 \\ + 0110 \\ \hline 110101 \end{array}$$

adding 6 (0110) to (ii) invalid part.

$$\begin{array}{r} 1101 \\ + 0110 \\ \hline 10011 \end{array}$$

Adding Carry to next 4-bit.

$$\begin{array}{r} 1101 \\ + 1 \\ \hline 1110 \end{array} > 9 \text{ which is invalid.}$$

Again, Add 6 (0110)

$$\begin{array}{r} 1110 \\ + 0110 \\ \hline 1)0100 \end{array} < 9 \text{ which is valid.}$$

Adding carry with next 4 bit.

$$\begin{array}{r} 0000 \\ + 1 \\ \hline 0001 \end{array} \text{ which is valid}$$

Hence,

$$\begin{array}{ccc} 0001 & 0100 & 0101 \\ 1 & 4 & 5 \end{array}$$

$$= (145)_{10}$$

Answer

Que. 199
+198

$$\begin{array}{r} 1001 \quad 1001 \\ 1001 \quad 1001 \\ \hline \end{array}$$

$$10010 \quad 10010 \leftarrow \text{invalid bit}$$

$$\begin{array}{r} 10010 \\ + 1010 \\ \hline 1)10000 \end{array}$$

$$\begin{array}{r} 10010 \\ + 1 \\ \hline 10011 \leftarrow \text{invalid} \end{array}$$

$$\begin{array}{r} 10011 \\ + 0110 \\ \hline 1)1001 \end{array} \text{ valid}$$

$$\begin{array}{r} 0000 \\ + 4 \\ \hline 0001 \end{array}$$

Hence,

$$\begin{array}{ccc} 0001 & 1001 & 1000 \\ 1 & 9 & 8 \end{array}$$

$$(198)_{10}$$

eg. (E) $945 + 278$

$$\begin{array}{r}
 1001 \\
 + 0010 \\
 \hline
 1011
 \end{array}
 \quad
 \begin{array}{r}
 0100 \\
 + 0111 \\
 \hline
 1011
 \end{array}
 \quad
 \begin{array}{r}
 0101 \\
 + 1000 \\
 \hline
 1101
 \end{array}$$

(i) (ii) (iii)

Here, all parts (i), (ii) & (iii) are invalid.

Now,

Adding 6 (0110) to first part

$$\begin{array}{r}
 1101 \\
 + 0110 \\
 \hline
 10011
 \end{array}$$

Adding carry to next bit.

$$\begin{array}{r}
 1011 \\
 + 1 \\
 \hline
 1100
 \end{array}$$

1100 ← invalid

Again

Add 6 (0110)

$$\begin{array}{r}
 1100 \\
 + 0110 \\
 \hline
 10010
 \end{array}$$

Adding carry to next bit

$$\begin{array}{r}
 1011 \\
 + 1 \\
 \hline
 1100
 \end{array}$$

$$\begin{array}{r}
 1100 \\
 + 1 \\
 \hline
 1100
 \end{array}$$

$$\begin{array}{r}
 1100 \\
 + 1 \\
 \hline
 1100
 \end{array}$$

← invalid

Again add 6 (0110)

$$\begin{array}{r}
 1100 \\
 + 0110 \\
 \hline
 10010
 \end{array}$$

$$\begin{array}{r}
 1100 \\
 + 0110 \\
 \hline
 10010
 \end{array}$$

$$\begin{array}{r}
 1100 \\
 + 0110 \\
 \hline
 10010
 \end{array}$$

Adding carry to next bit

$$\begin{array}{r}
 0000 \\
 + 1 \\
 \hline
 0001
 \end{array}$$

$$\begin{array}{r}
 0000 \\
 + 1 \\
 \hline
 0001
 \end{array}$$

which is valid.

Hence,

$$\begin{array}{cccc}
 0001 & 0010 & 0010 & 0011 \\
 1 & 2 & 2 & 3
 \end{array}$$

$$(1223)_{10}$$

eg. (8) $945 + 278$

1001	0100	0101
+ 0010	0111	1000
1011	1011	1101
(i)	(ii)	(iii)

Here, all parts (i), (ii) & (iii) are invalid.

Now,

Adding 6 (0110) to first part

$$\begin{array}{r} 1101 \\ + 0110 \\ \hline 10011 \end{array}$$

Adding carry to next bit.

1011

+ 1

1100 ← invalid

Again

Add 6 (0110)

1100

+ 0110

10010

Adding carry to next bit

1011

+ 1

1100

← invalid

Again add 6 (0110)

1100

+ 0110

10010

Adding carry to next bit

0000

+ 1

0001

which is valid.

Hence,

0001 0010 0010 0011

1

2

2

3

$(1223)_{10}$

Gray Code (Reflected Code):

It is an ordering of Binary number system such that two successive values differ in only one bit.

It was named after Frank Gray.

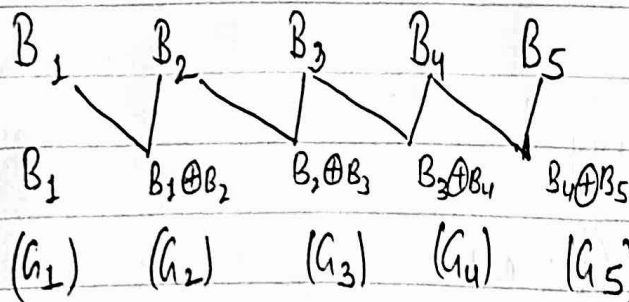
(A) Binary to Gray Conversion:-

In this first convert the number into its equivalent binary form then write 1st bit as it is and 2nd bit is EX-OR additional of 1st and 2nd bit then 3rd bit is EX-OR addition of 2nd and 3rd bit & so on.

Binary	Gray
000	000
001	001
010	011
011	010
100	110
101	111
110	101
111	100

only 1 bit changed

EX-OR		Addition
A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



eg. $(27)_{10} \xrightarrow{\text{binary}} (11011)_2$
 $\downarrow$
 $(10110)_{\text{gray}}$

2 | 27 - 1
 2 | 13 - 1
 2 | 6 - 0
 2 | 3 - 1
 2 | 1 - 1
 0

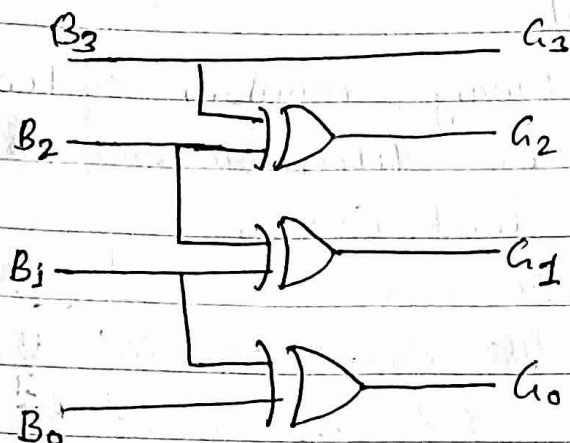


Fig. Logical Diagram of ^{4-bit} Binary to gray Conversion.

Conversion from BCD to Gray:-

Step 1:-

Most Significant Bit of the Gray Code & the BCD is same.

Step 2:-

Going from left to right add each adjacent pair, to get next gray code.

Discard the carry.

Example:-

(a) $(0010)_2 = (\quad)_{\text{gray}}$

$(0 \oplus 0 \oplus 1 \oplus 1)_{\text{gray}}$

(b) $(4593)_{10} = (\text{gray code})$

$(4593)_{10} = (1000111110001)_2$

$= 1100100001001$

$$\begin{array}{r}
 2 \overline{) 4593} \text{ --- } 1 \\
 2 \overline{) 2296} \text{ --- } 0 \\
 2 \overline{) 1148} \text{ --- } 0 \\
 2 \overline{) 574} \text{ --- } 0 \\
 2 \overline{) 287} \text{ --- } 1 \\
 2 \overline{) 143} \text{ --- } 1 \\
 2 \overline{) 71} \text{ --- } 1 \\
 2 \overline{) 35} \text{ --- } 1 \\
 2 \overline{) 17} \text{ --- } 1 \\
 2 \overline{) 8} \text{ --- } 0 \\
 2 \overline{) 4} \text{ --- } 0 \\
 2 \overline{) 2} \text{ --- } 0
 \end{array}$$

$$\textcircled{c} (1000)_2 = ()_{\text{gray code}}.$$

$$= (1100)_{\text{gray code}}.$$

$$\textcircled{d} (49)_{10} = ()_{\text{gray code}}$$

Sol. Decimal $\rightarrow$ binary $\rightarrow$ gray code.

$$()_{10} \rightarrow ()_2 \rightarrow ()_{\text{gray code}}.$$

$$(49)_{10} = (110001)_2$$

$$\downarrow$$

$$= (101001)_{\text{gray}}$$

$$\begin{array}{r} 2 \overline{) 49} - 1 \\ 2 \overline{) 24} - 0 \\ 2 \overline{) 12} - 0 \\ 2 \overline{) 6} - 0 \\ 2 \overline{) 3} - 1 \\ 2 \overline{) 1} - 1 \\ 0 \end{array} \uparrow$$

$\textcircled{e}$ Convert $(11101)_2$ to gray code

$$\begin{array}{cccccc} 1 & 1 & 1 & 0 & 1 \\ & \swarrow \oplus & \swarrow \oplus & \swarrow \oplus & \swarrow \oplus \\ 1 & 0 & 0 & 1 & 1 \end{array}$$

$$(11101)_2 = (10011)_{\text{gray code}}.$$

Que: i) $(54)_{10} = ()_{\text{gray code}}$

ii) $(27)_{10} = ()_{\text{gray code}}$

iii) $(11000110)_2 = ()_{\text{gray code}}.$

iv) $(27)_{10} = ()_{\text{gray code}}$

v) $(49)_{10} = ()_{\text{gray code}}$

③ Gray to binary Conversion: -

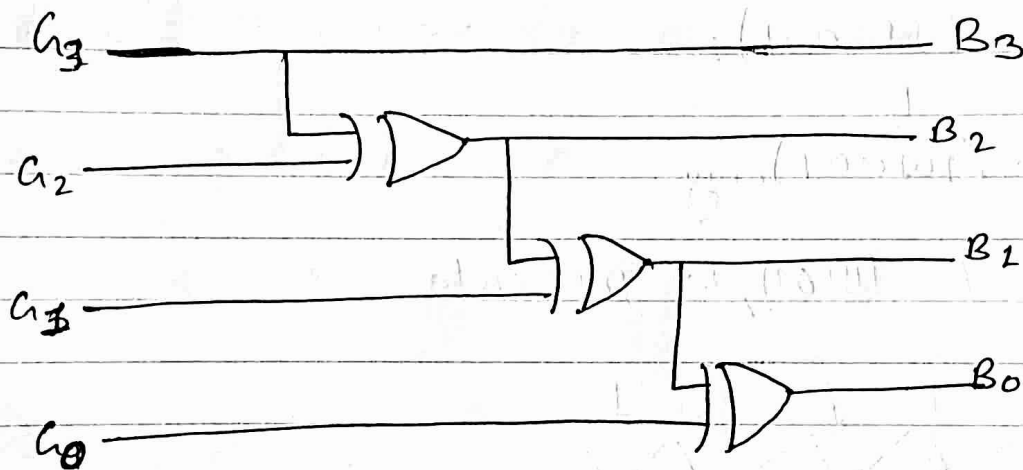
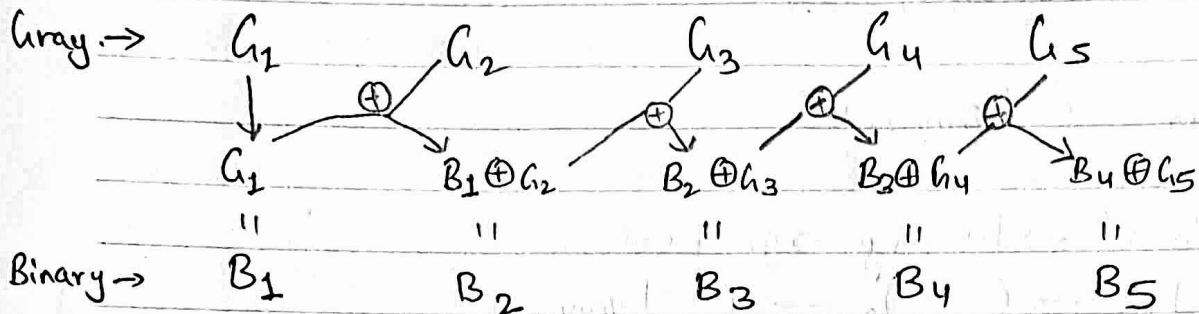
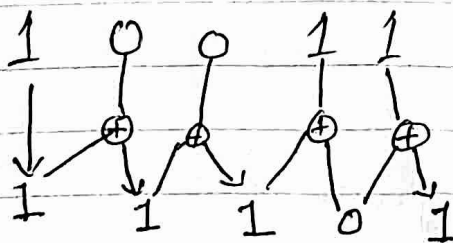


Fig:- logical Diagram of 4-bit Gray to Binary Conversion.

Example:- Convert 10011 to binary.



Ques:-

$$\textcircled{a} (10101111)_{\text{Gray}} = (\quad)_2$$

$$(10011)_{\text{Gray}} = (11101)_2$$

Excess-3 Code (Xs-3) :-

In this, first we find BCD of each decimal digit then add 3 to each decimal value.

It is an important 4-bit code sometimes used with binary-coded decimal (BCD) numbers. To convert any decimal number into its excess-3 form, we have to add 3 to each decimal digit and then convert the sum to a BCD Number.

eg.

$(1532)_{10}$

$$\begin{array}{cccc}
 0001 & 0101 & 0011 & 0010 \\
 + 0011 & 0011 & 0011 & 0011 \\
 \hline
 0100 & 1000 & 0110 & 0101
 \end{array}$$

Xs-3

Example :-

(a) Convert 12 to an excess-3 number.

First add 3 to each decimal digit. ie.

$$\begin{array}{cc}
 1 & 2 \\
 + 3 & + 3 \\
 \hline
 4 & 5
 \end{array}$$

Second, Convert the sum to BCD form.

$$\begin{array}{cc}
 4 & 5 \\
 \downarrow & \downarrow \\
 0100 & 0101
 \end{array}$$

$\therefore 01000101$ is the excess-3 code stands for decimal 12.

Date _____ Page 48

⑥ Convert 29 to an excess-3 code.

$$\begin{array}{r}
 2 \quad 9 \\
 + 3 \quad + 3 \\
 \hline
 5 \quad 12 \\
 \downarrow \quad \downarrow
 \end{array}$$

0101 1100

After adding 3 and 3 do not carry the 1 into the next column; instead leave the result intact as 12, and then convert. Therefore 01011100 is excess-3 code stands for decimal 29.

⑦ $(25.35)_{10}$

$$\begin{array}{r}
 25.35 \\
 + 33.33 \\
 \hline
 (58.68)_{10}
 \end{array}$$

 $(01011000.01101000)_{XS-3}$ ⑧ $(22)_{10} \rightarrow (\text{?})_{\text{excess-3 code}}$

$$\begin{array}{r}
 22 \\
 + 33 \\
 \hline
 55
 \end{array}
 \quad
 \begin{array}{r}
 0010 \quad 0010 \\
 0011 \quad 0011 \\
 \hline
 0101 \quad 0101
 \end{array}$$

$\therefore (01010101)_{\text{excess-3 code of } (22)_{10}}$

imp.
 Convert $(47.3)_7$ into BCD, ~~Ex~~ Excess-3 & Gray Code.
 Sol.

Convert base 7 to decimal

$$(47.3)_7$$

$$(4 \times 7^1) + (7 \times 7^0) + (3 \times 7^{-1})$$

$$= 28 + 7 + 0.42857$$

$$= (35.42857)_{10}$$

i) BCD

$$(35.42857)_{10}$$

$$(0011 \ 0101 \ . \ 0100 \ 0010 \ 1000 \ 0101 \ 0111)_{BCD}$$

ii) Excess-3

$$35.42857$$

~~$$(35.42857)_{10}$$~~

$$\begin{array}{ccccccc} 0011 & 0101 & . & 0100 & 0010 & 1000 & 0101 & 0111 \\ 0011 & 0011 & . & 0011 & 0011 & 0011 & 0011 & 0011 \\ \hline (0110 & 1000 & . & 0111 & 0101 & 1011 & 1000 & 1010)_{XS-3} \end{array}$$

iii) Gray code $(35.4285)_{10} = (100011.01101)_2$

$$\begin{array}{ccccccc} 1 & 0 & 0 & 0 & 1 & 1 & . & 0 & 1 & 1 & 0 & 1 \\ \downarrow & \oplus & \oplus & \oplus & \oplus & \oplus & & \oplus & \oplus & \oplus & \oplus & \\ (1 & 1 & 0 & 0 & 1 & 0 & . & 1 & 1 & 0 & 1 & 1)_{Gray\ code} \end{array}$$

Error-detection Codes:-

- * Single bit error eg. 101 → 100 or 1 → 0
 5 4 0 → 1

- * Burst Error (more than one bit error)



length of error $\rightarrow 5$ bit

- * If the bandwidth of a channel is 1 Mbps then for how much duration the error should last?



$$1 \text{ Gbps} = 10^9 \text{ bits/sec.}$$

$$1 \text{ bits} = \frac{1}{10^9} \text{ sec}$$

Detection Method

- Simple parity (Even, ODD)
- 2D parity check
- Check Sum
- CRC (cyclic Redundancy Check)

Correction

- Hamming Codes

message bits.

Date

Page 51

Single parity :-

→ $m+1$ bits.

→ Even parity (no. of '1' should be even)

eg. 1010 → 0

1110 → 1

→ Can detect all single bit errors in code word

eg. 11101

↓

01101 ← parity bit

→ Can detect all odd no. of errors also

11101 → 00101 ← parity bit

Parity Method for Error detection :-

3 bit data			Message with even parity		Message with odd parity	
A	B	C	Message	Parity	Message	Parity
0	0	0	000	0	000	1
0	0	1	001	1	001	0
0	1	0	010	1	010	0
0	1	1	011	0	011	1
1	0	0	100	1	100	0
1	0	1	101	0	101	1
1	1	0	110	0	110	1
1	1	1	111	1	111	0

Note :-

① even parity : No. of '1' should be even then parity

② Odd parity : will be 0

No. of '1' should be odd then parity will be 0.

ASCII Code :-

~~American Standard Code for Information Interchange~~

Binary information can be transmitted from one location to another by electric wires or other communication medium.

Any external noise introduced into the physical communication medium may change some of the bits from 0 to 1 or vice versa.

- The purpose of an error detection code is to detect such bit reversal errors.
- One of the most common ways to achieve error detection is by means of a parity bit.
- A parity bit is a check bit, which is added to a block of data for error detection purpose.
- Parity check is suitable for single bit error detection only.

There are two types of parity checking

① Even parity

② Odd Parity.

Even		Odd	
Message	Parity (P)	Message	Parity (P)
0000	0	0000	1
0001	1	0001	0
0010	1	0010	0
0011	0	0011	1
0100	1	0100	0
0101	0	0101	1
0110	0	0110	1
0111	1	0111	0
1000	1	1000	0
1001	0	1001	1
1010	0	1010	1
1011	1	1011	0
1100	0	1100	1
1101	1	1101	0
1110	1	1110	0
1111	0	1111	1

Error Checking Mechanism:-

→ During the transmission of information from one location to another, an even parity bit is generated in the sending end for each message transmission. The message together with the parity bit, is transmitted to its destination. The parity of the received data is checked in the receiving end. If the parity of the received information is not even, it means that at least one bit has changed value during the transmission.

- This method detects one, three, or any odd combination of errors in each message that is transmitted.
- An even combination of ~~even~~ errors is undetected. Additional error-detection schemes may be needed to take care of an even combination of errors.

ASCII Code:-

(American Standard Code for Information interchange)

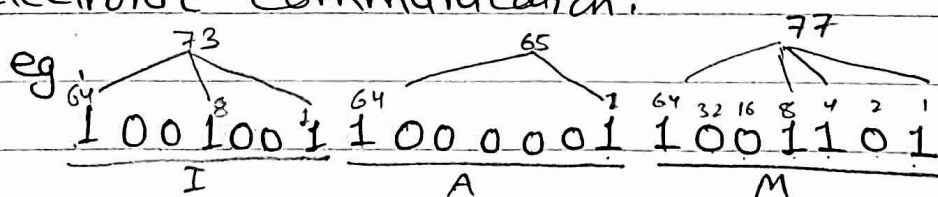
- First 31 (0-31) is for Control Character (unprintable) are used to control peripherals such as printers.
- Codes 32-127 are for printable character represent letters digits, punctuation marks and some symbols.

32	space	42	*
33	!	43	+
34	"	44	,
35	#	45	-
36	\$	46	.
37	%	47	/
38	&	48-57	(0-9)
39	'	58	:
40	(	59	;
41	)	60	<

61	=	123	{
62	>	124	
63	?	125	}
64	@	126	~ (equivalent)
65-90	(A-Z)	127	Delete
91	[		
92	\		
93	]		
94	^		
95	_ (underscore)		
96	` (grave accent)		
97-122	(a-z)		

→ ASCII Code is 7-bit code. So, it can represent 128 characters.

→ It is a character encoding standard for electronic communication.



IAM is ASCII code.

→ 7 bit character code. It range from 0-127 $[(2^7) = 128]$